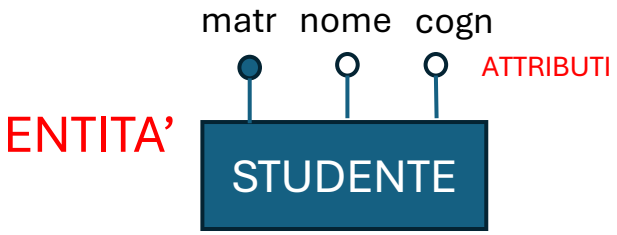
MONDO DELLE BASI DI DATI



MONDO DELL'OOP

CLASSE

INCAPSULAMENTO di
- Dati
- Operazioni sui dati

ASTRAZIONE

STRUTTURA DEI DATI

ATTRIBUTI/ **PROPRIETA'**

FUNZIONI/ **METODI**

Studente
matricola nome cognome
__init__() def sostiene_esame() ...

PYTHON

```
class Studente:
    matr = 0
    nome = ""
    cognome = ""
    def __init()__:
        ....
```

SCHEMA/DIAGRAMMA E-R

UML - DIAGRAMMA DELLE CLASSI

REIFICAZIONE
CREO UN OGGETTO/ UN'ISTANZA

matricola	nome	cognome
12345	Mario	Rossi
67890	Gianni	Verdi

TUPLA/ISTANZA

DB RELAZIONALE (MYSQL, ORACLE, ...)

DATI VERI E PROPRI

s:Studente
matricola = 123456 nome = "Mario" cognome = "Rossi"
__init__() def sostiene_esame() ...

OGGETTO/ISTANZA

PYTHON

```
s = Studente()
s = Studente(123456,
             "Mario",
             "Rossi")

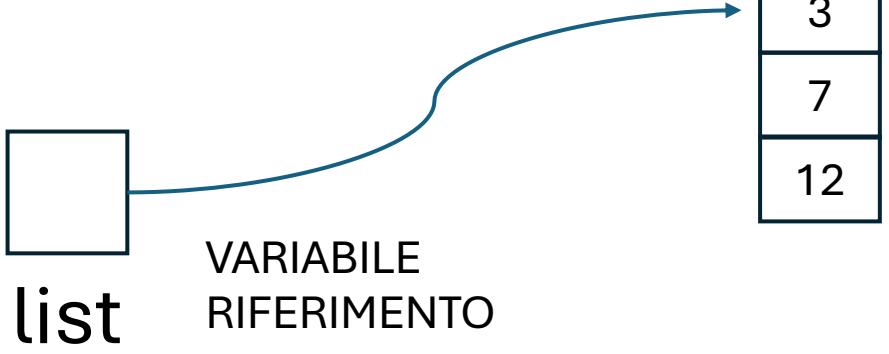
s.matricola = 77777
print(s.nome)
s.sostiene_esame()
```

x = 5

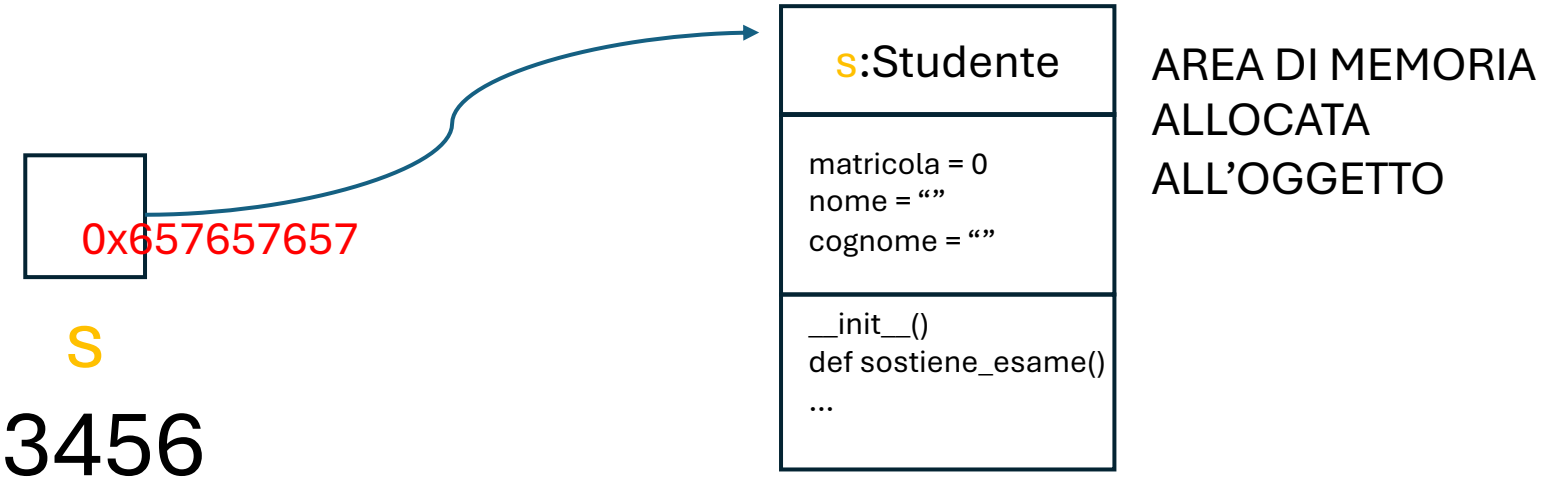


x

list = [3, 7, 12]



s = Studente()

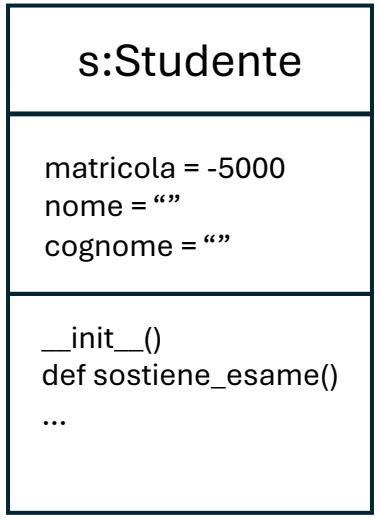


s.matricola = 123456

s.sostiene_esame()

CON LA NOTAZIONE PUNTATA, OVVERO IL ., POSSO ACCEDERE A CIO’ CHE STA NELLA “PANCIA” DELL’OGGETTO s (ATTRIBUTI, IN LETTURA E SCRITTURA, E FUNZIONI)

OK, LA CLASSE E' UN NUOVO MODO DI **MODULARIZZARE** IL CODICE



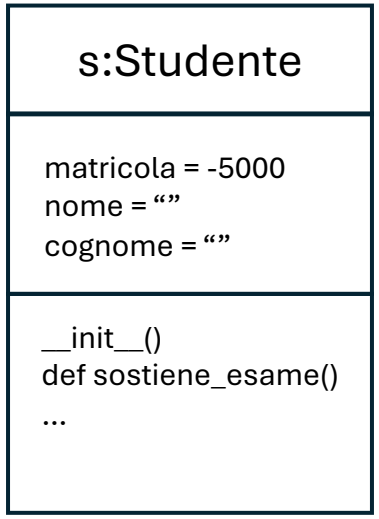
PYTHON

```
class Studente:
    self.matr = 0
    self.nome = ""
    self.cognome = ""
    def __init()__:
        ....
```

OK, LA CLASSE E' UN NUOVO MODO DI **MODULARIZZARE** IL CODICE

PERO' UN'ALTRA COSA CHE CI PIACEREBBE E' CONTROLLARE COME AVVENGANO GLI ACCESSI ALLA CLASSE/OGGETTO ED AI SUOI ATTRIBUTI/METODI (NO "SPAGHETTI CODE")"

IN PRATICA, VORREI UN MODO PER CONTROLLARE LA VISIBILITA' DI ATTR./METODI (MEMBRI DELLA CLASSE)



PYTHON

```
class Studente:
    self._matr = 0
    self.__nome = ""
    self.cognome = ""
    def __init()__:
      ....
```

UN SOLO _ E' UNA INDICAZIONE

DUE _ INVECE (__) PORTANO PYTHON A CAMBIARE PROPRIO IL NOME DELL'ATTRIBUTO, NON PIU' RAGGIUNGIBILE (SE NON CON ISTRUZIONI COMPLESSE)

SE SCRIVESSI s._matr = 5 # WARNING

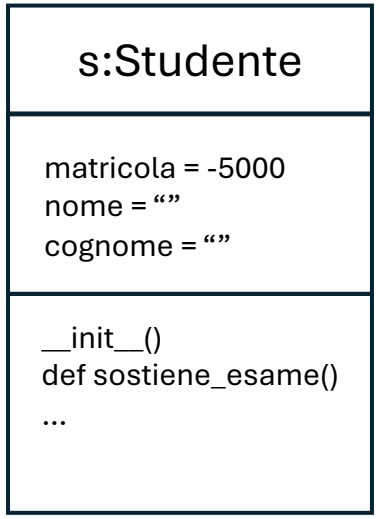
SE SCRIVESSI s.__nome = "Mario" # ERRORE

OK, LA CLASSE E' UN NUOVO MODO DI **MODULARIZZARE** IL CODICE

PERO' UN'ALTRA COSA CHE CI PIACEREBBE E' CONTROLLARE COME AVVENGANO GLI ACCESSI ALLA CLASSE/OGGETTO ED AI SUOI ATTRIBUTI/METODI (NO "SPAGHETTI CODE")"

IN PRATICA, VORREI UN MODO PER CONTROLLARE LA VISIBILITA' DI ATTR./METODI (MEMBRI DELLA CLASSE)

L'INSIEME DEI METODI PUBBLICI DI UNA CLASSE E' CHIAMATO **INTERFACCIA**



PYTHON

```
class Studente:
    self.__matr = 0
    self.__nome = ""
    self.__cognome = ""
    def __init()__:
    def sostiene_esame()
```

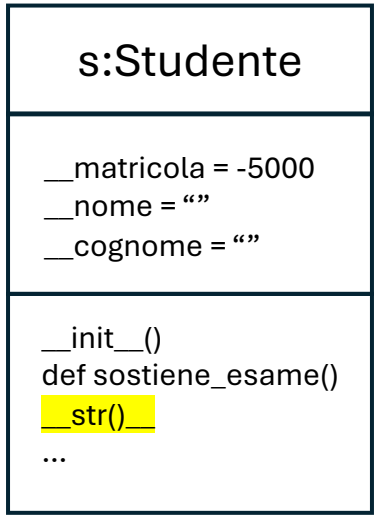
UNA BUONA REGOLA DI PROGRAMMAZIONE AD OGGETTI E' QUELLA DI DEFINIRE GLI ATTRIBUTI CON _ _ (NASCOSTI/PRIVATI) ...

... E METTERE A DISPOSIZIONE METODI NON NASCOSTI (PUBBLICI) CON CUI ACCEDERE AD ATTRIBUTI ED EFFETTUARE ALTRE OPERAZIONI SULLA CLASSE/OGGETTO

INFORMATION HIDING
ALTRO MECCANISMO DELLA PROGRAMMAZIONE A OGGETTI PER RENDERE IL CODICE PIU' ROBUSTO

DELEGATION: ALTRO CONCETTO FONDAMENTALE DELLA OOP

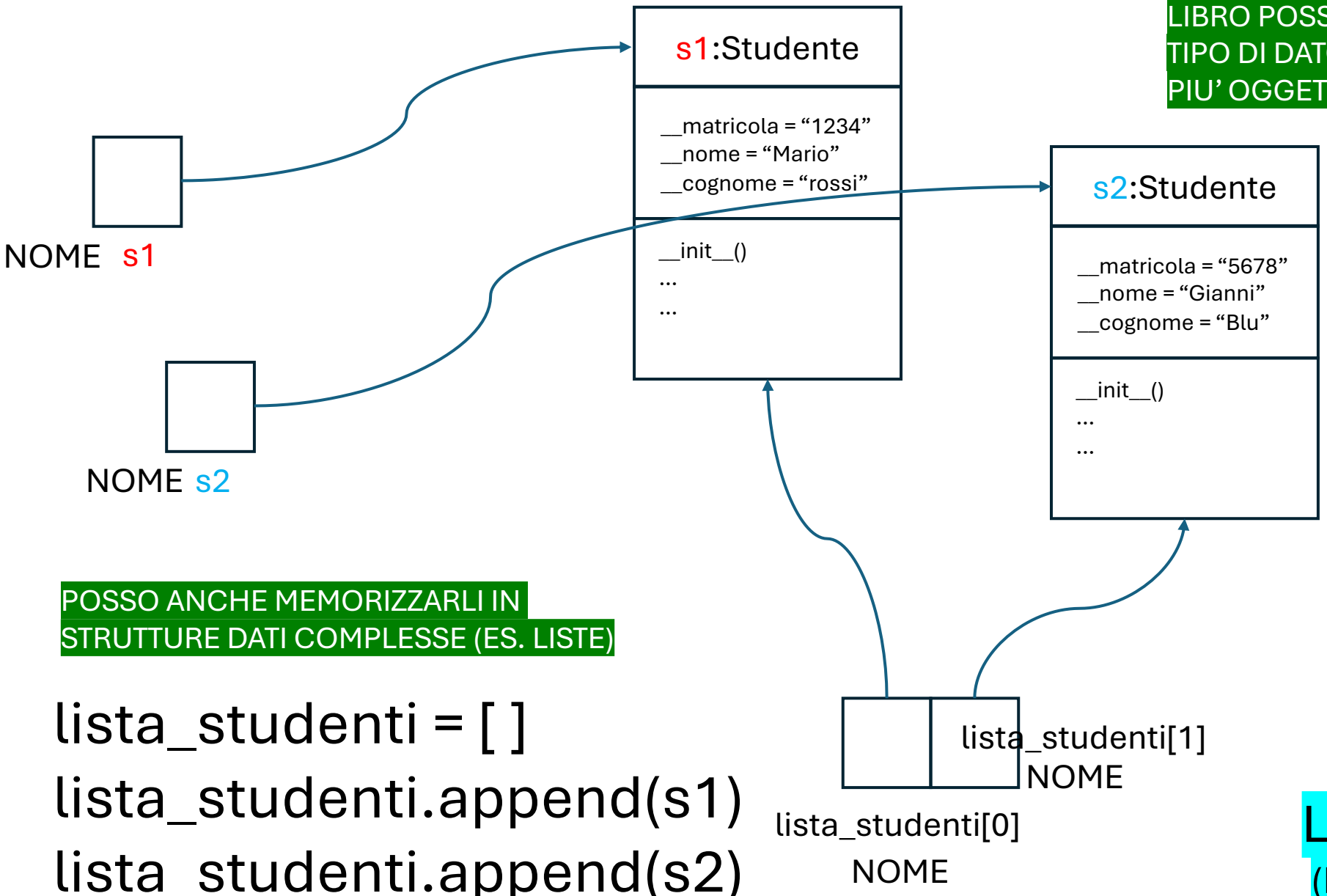
IL METODO __str()__ E' IL MODO STANDARD DI UN OGGETTO PYTHON DI DESCRIVERSI SOTTO FORMA DI STRINGA



ANZICHE' __str()__ POTREI CHIAMARLO descriviti_come_stringa()

QUANDO DEVO STAMPARE UN OGGETTO NELLA CONSOLE, ANZICHE' ANDARE IO AD ACCEDERE AI SINGOLI ATTRIBUTI (CHE TRA L'ALTRO POTREBBERO/DOVREBBERO ESSERE PRIVATI) PERCHE' NON **DELEGARE** ALL'OGGETTO IL COMPITO DI FARLO (CHI MEGLIO DI UNO Studente SA COME STAMPARSI/DESCRIVERSI)

DOPO CHE ABBIAMO CREATO LA CLASSE LIBRO POSSO USARLA COME UN NUOVO TIPO DI DATO DEL LINGUAGGIO, E CREARE PIU' OGGETTI DI QUEL TIPO

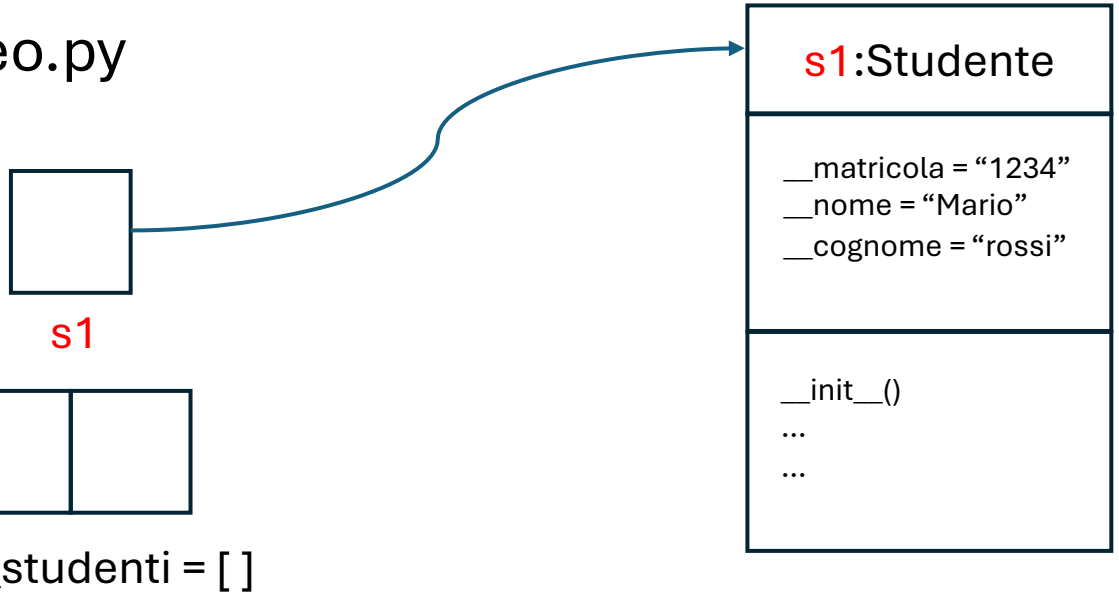


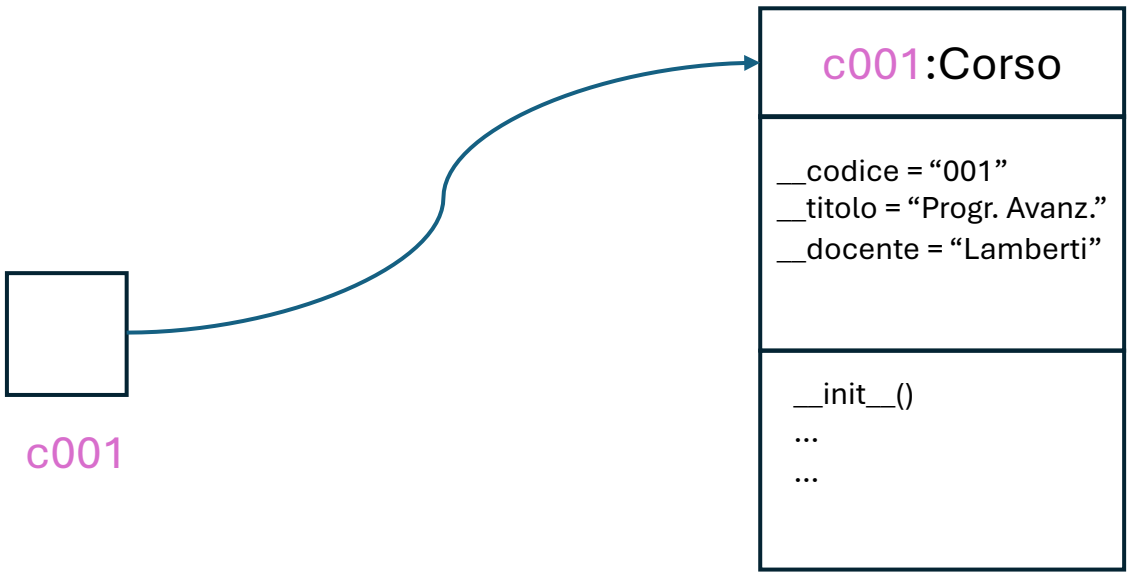
POSSO ANCHE MEMORIZZARLI IN STRUTTURE DATI COMPLESSE (ES. LISTE)

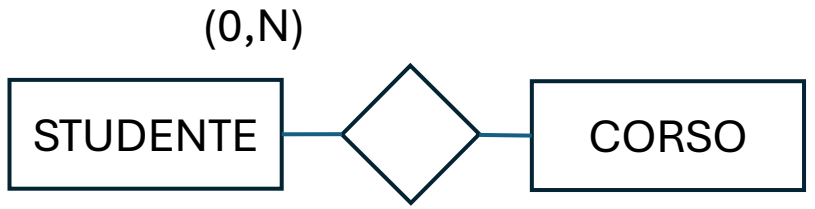
```
lista_studenti = [ ]
lista_studenti.append(s1)
lista_studenti.append(s2)
```

LISTA DI OGGETTI
(E ALIASING, PIU' NOMI PER LO STESSO OGGETTO)

ateneo.py







Iscritto a

RELAZIONE

COME FACCIO AD ESPRIMERE
LE RELAZIONI TRA OGGETTI?

ESEMPIO, IL CONCETTO CHE UNO
STUDENTE “E’ ISCRITTO” AD UN CORSO?

MONDO DELLE BASI DI DATI

ateneo.py



s1



lista_studenti = [ ]

s1:Studente

__matricola = “1234”
__nome = “Mario”
__cognome = “rossi”
__iscrittoa=“001”
__titolo=“Progr. Avanz”
__docente=“Lamberti”

__init__()
…
…

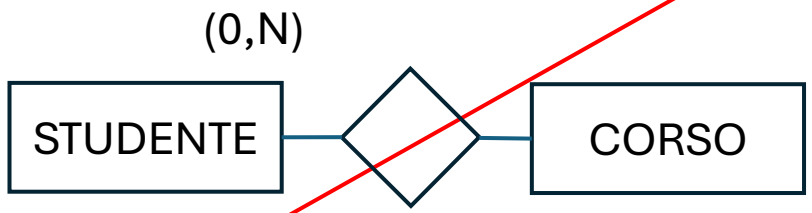


c001

c001:Corso

__codice = “001”
__titolo = “Progr. Avanz.”
__docente = “Lamberti”

__init__()
…
…



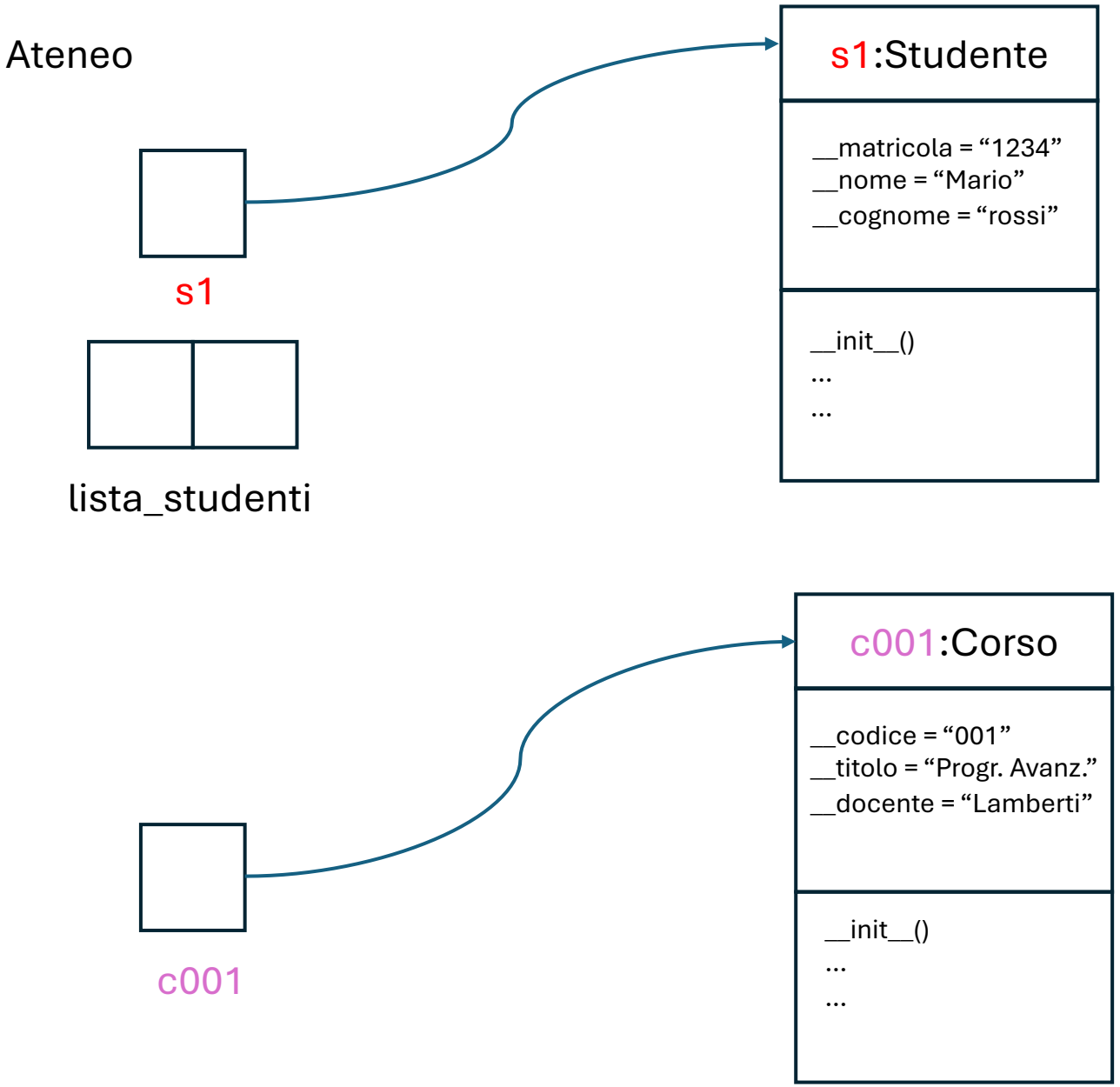
RELAZIONE

UN MODO PER FARLO POTREBBE ESSERE INSERIRE LE INFORMAZIONI RELATIVE AL Corso A CUI LO Studente E’ ISCRITTO DIRETTAMENTE NELLO Studente (COPIO LE INFORMAZIONI)

COME FACCIO AD ESPRIMERE LE RELAZIONI TRA OGGETTI?

ESEMPIO, IL CONCETTO CHE UNO STUDENTE “E’ ISCRITTO” AD UN CORSO?

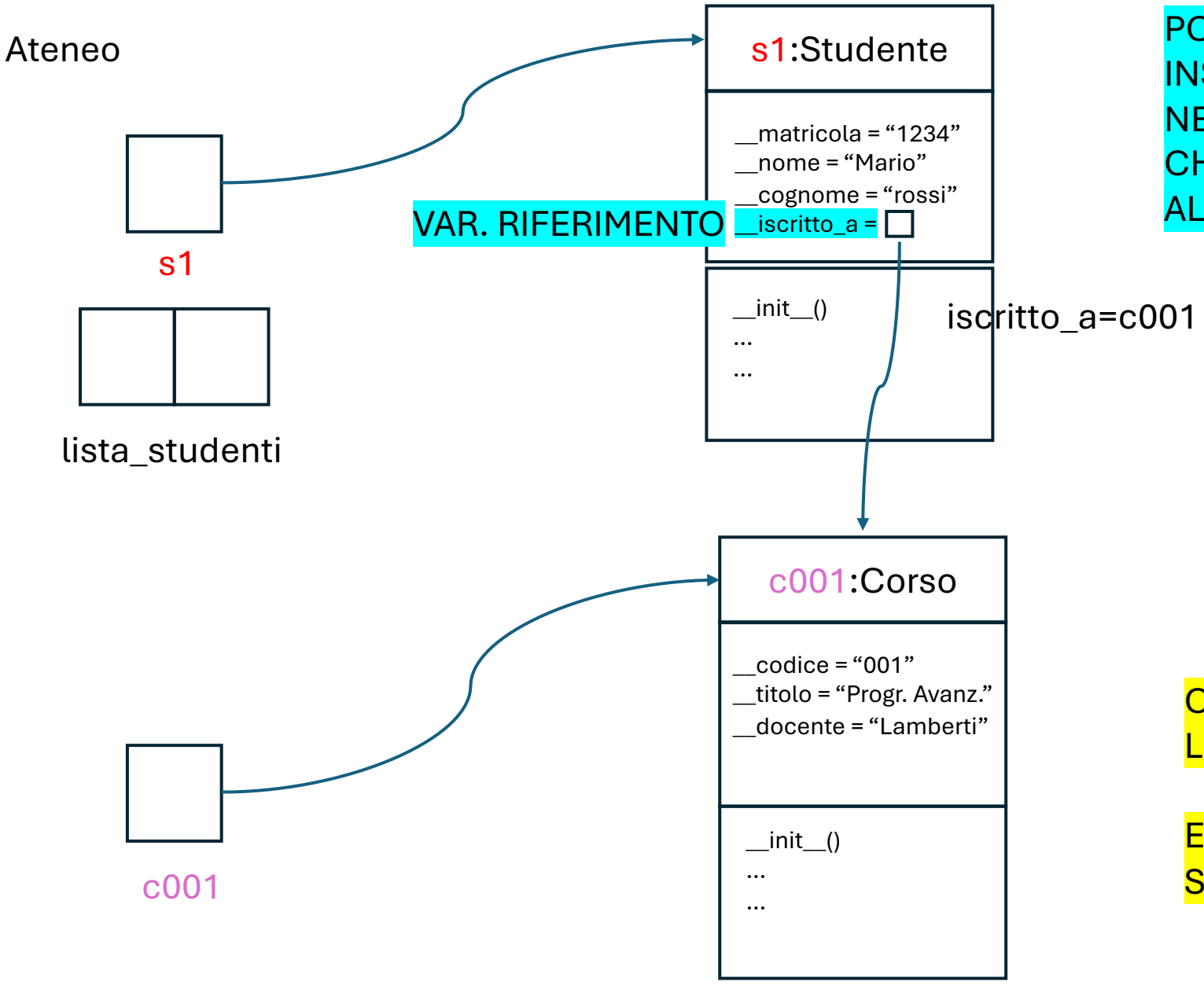
MAI UNA COSA BUONA, GIA’ DAL MONDO DELLE BASI DI DATI, FONTE DI POSSIBILI INCONSISTENZE



NEL MONDO DELLE BASI DI DATI AVREI CREATO DELLE CHIAVI ESTERNE CHE COLLEGAVANO TRA LORO LE TABELLE

COME FACCIO AD ESPRIMERE LE RELAZIONI TRA OGGETTI?

ESEMPIO, IL CONCETTO CHE UNO STUDENTE “E’ ISCRITTO” AD UN CORSO?

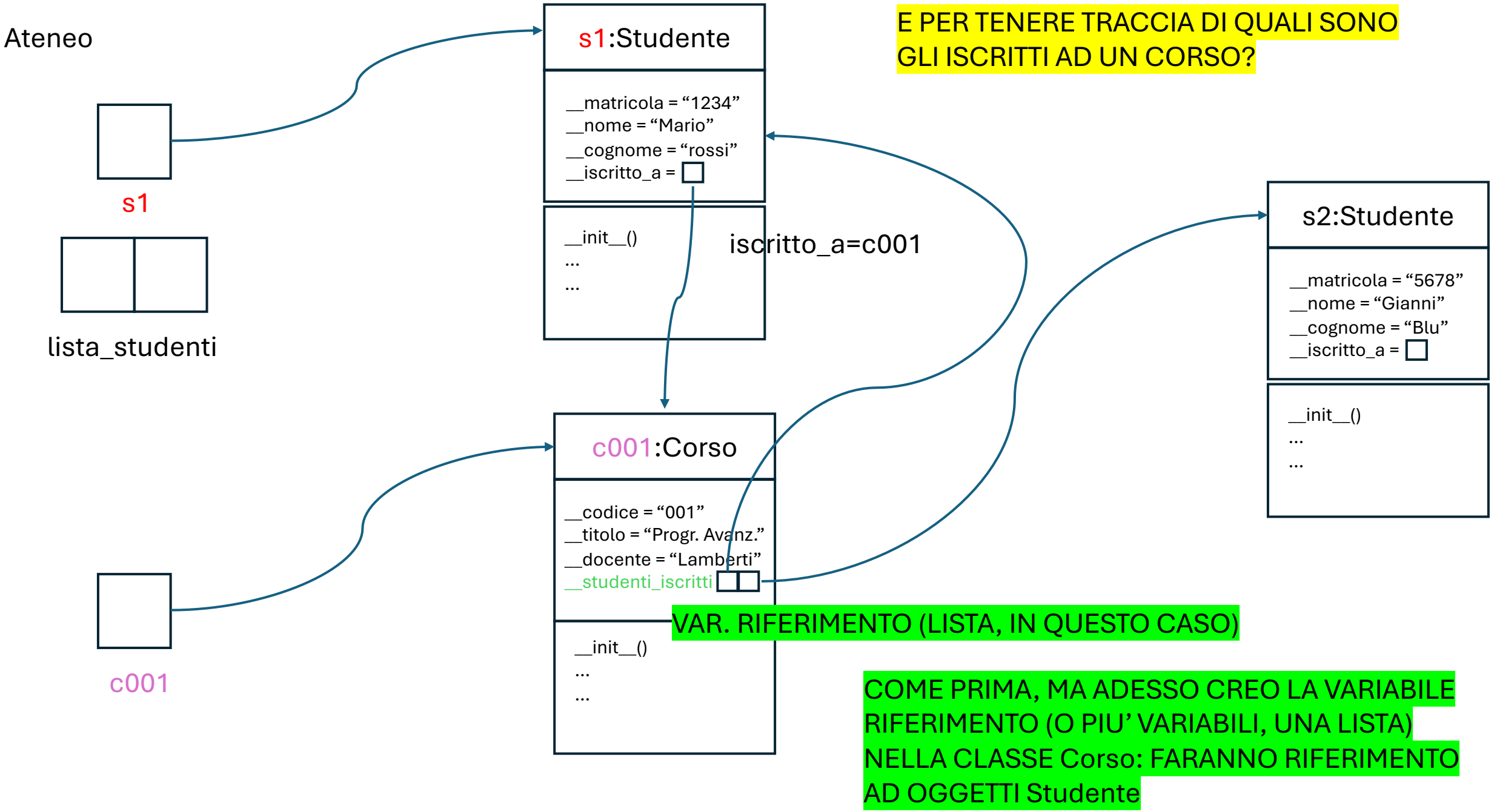


POSSO FARE QUALCOSA DI SIMILE, INSERENDO UN NUOVO ATTRIBUTO NELLA CLASSE Studente, CHE MEMORIZZI IL RIFERIMENTO AL Corso (SENZA DUPLICARE DATI)

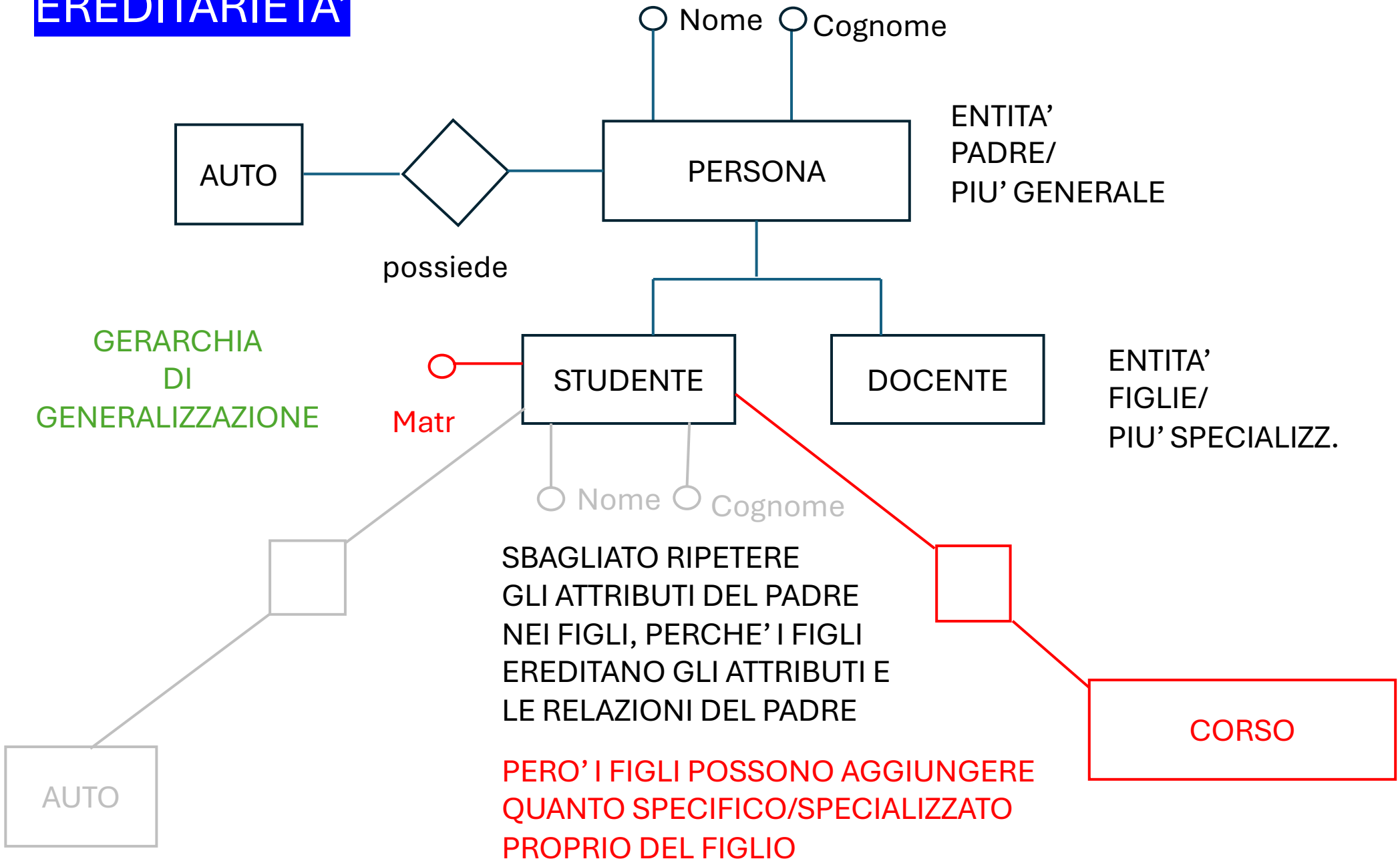
NEL MONDO DELLE BASI DI DATI AVREI CREATO DELLE CHIAVI ESTERNE CHE COLLEGAVANO TRA LORO LE TABELLE

COME FACCIO AD ESPRIMERE LE RELAZIONI TRA OGGETTI?

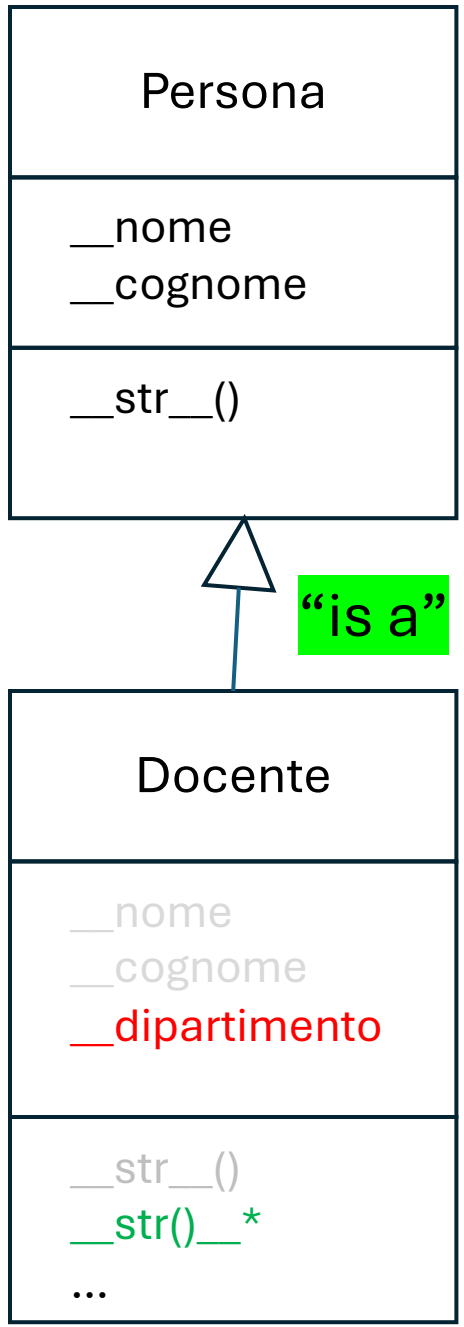
ESEMPIO, IL CONCETTO CHE UNO STUDENTE “E’ ISCRITTO” AD UN CORSO?


Ateneo
s1
lista_studenti
s1:Studente
__matricola = “1234”
__nome = “Mario”
__cognome = “rossi”
__iscritto_a =
__init__()
...
...
iscritto_a=c001
E PER TENERE TRACCIA DI QUALI SONO GLI ISCRITTI AD UN CORSO?
s2:Studente
__matricola = “5678”
__nome = “Gianni”
__cognome = “Blu”
__iscritto_a =
__init__()
...
...
c001:Corso
__codice = “001”
__titolo = “Progr. Avanz.”
__docente = “Lamberti”
__studenti_iscritti
VAR. RIFERIMENTO (LISTA, IN QUESTO CASO)
__init__()
...
...
c001
COME PRIMA, MA ADESSO CREO LA VARIABILE RIFERIMENTO (O PIU’ VARIABILI, UNA LISTA) NELLA CLASSE Corso: FARANNO RIFERIMENTO AD OGGETTI Studente

EREDITARIETA'



CLASSE
PADRE/
GENERALE/
DI BASE/
ANTENATO



SI DICE CHE Docente ESTENDE Persona,
Docente IS A Persona

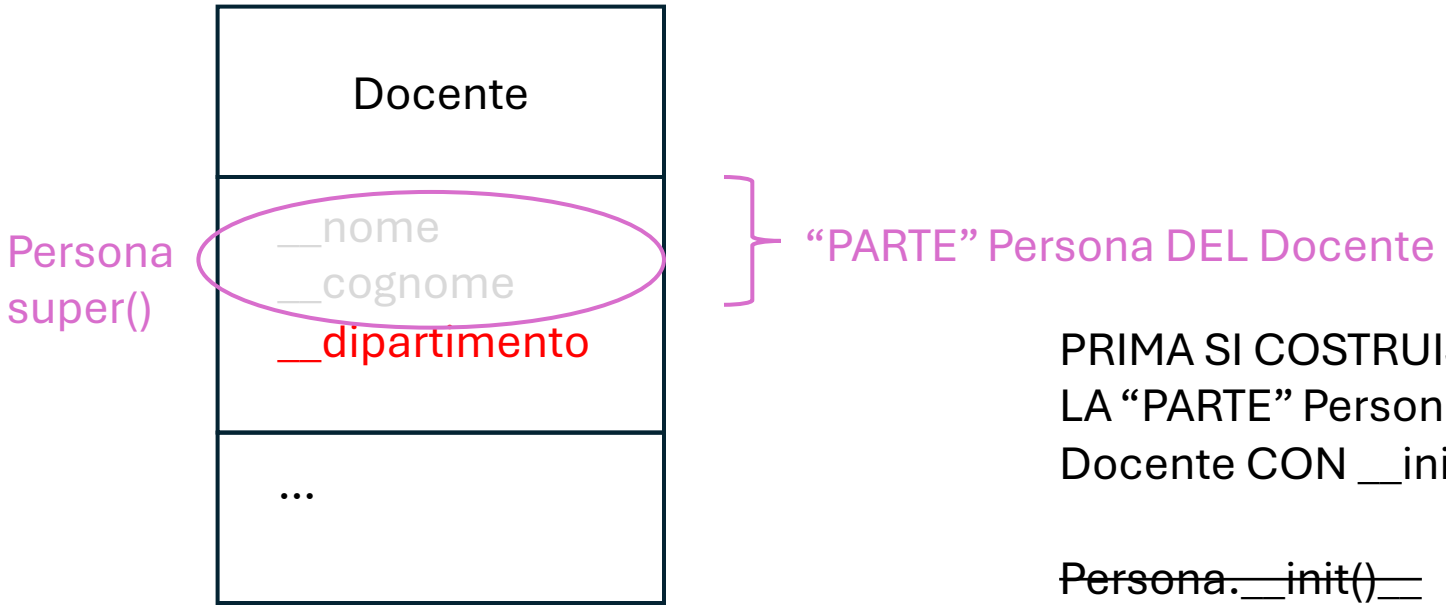
CLASSE
FIGLIA/
PIU' SPECIFICA/
DERIVATA

LA CLASSE Docente CHE **DERIVA DA** Persona **EREDITA**
GLI ATTRIBUTI DI Persona (__nome e __cognome), E' QUINDI SBAGLIATO RIPETERLI

PUO' INOLTRE **AGGIUNGERE** NUOVI ATTRIBUTI, PIU' SPECIFICI (__dipartimento) ED ANCHE NUOVI METODI

MA, NOVITA' DELLA OOP, PUO' ANCHE **CAMBIARE CIO' CHE NON VA BENE**,
*RIDEFINENDOLO** (OVERRIDING)

COSTRUZIONE DI OGGETTI DI CLASSI FIGLIE, ES. Docente DA Persona, “A CIPOLLA”



PRIMA SI COSTRUISCE LA “PARTE” Persona DEL Docente CON __init()__

~~Persona.__init()__~~

SI USA LA PAROLA CHIAVE **super**, CHE SIGNIFICA L’OGGETTO/LA CLASSE DA CUI DERIVO

POI SI AGGIUNGE/INIZIALIZZA QUANTO SPECIFICO DEL DECENTE (es. dipartimento)

super().__init(<params>)__

ISTRUZIONE DA INSERIRE ALL’INIZIO DEL COSTRUTTORE DELLA CLASSE Docente, CHIAMA IL COSTRUTTORE DI Persona